Java SE 17 Programmer II（1Z0-826-JPN）　試験内容 1 コレクションとジェネリクス　#03

ソートと便利なメソッド

Comparable・Comparator と、Collections・Arrays の道具

各ページの下に、根拠にした資料（Oracle の試験内容チェックリスト、Java SE 17 の API ドキュメント、Java 言語仕様 JLS）を書いた。コードの結果は JDK 17（Eclipse Temurin 17.0.21）で実行して確かめた

全体像 2 つの比べ方

	Comparable<T>	Comparator<T>
パッケージ	java.lang	java.util
メソッド	int compareTo(T o)	int compare(T a, T b)
どこに書く	並べたいクラス自身（自然順序）	別のクラス・ラムダ式・メソッド参照
使い方	Collections.sort(list)・list.sort(null)・new TreeSet<>()	list.sort(c)・new TreeSet<>(c)・Arrays.sort(arr, c)

戻り値は「負・0・正」だけに意味がある。String は文字コードの差（"A".compareTo("a") は −32）、Integer は −1・0・1。

TreeSet・TreeMap は compare が 0 なら「同じ要素」。equals では違っても捨てられる。

出典：API Comparable・Comparator・String.compareTo・Integer.compareTo／The Java Tutorials「Object Ordering」

第1章

Comparable と自然順序

クラス自身が持つ、ただ 1 つの標準の並び順。

Comparable を実装する

```
record Item(String name, int price)
    implements Comparable<Item> {
  public int compareTo(Item o) {
    return Integer.compare(price, o.price);
  }
}
Collections.sort(list); // price の安い順
```

> Comparable でない要素の List を Collections.sort に渡すと**コンパイルエラー**、TreeSet に add すると**実行時に** ClassCastException。

equals と一貫していないと

```
pen(100)・cup(100)・box(50)
TreeSet → 2 個  // pen と cup は compareTo が 0
HashSet → 3 個  // equals は name も見る
```

null を含む List

```
Collections.sort([b, null, a]) // NullPointerException
nullsFirst(naturalOrder())    // [null, a, b]
nullsLast(reverseOrder())     // [b, a, null]
```

出典：API Comparable（consistent with equals）・Collections.sort・TreeSet・Comparator.nullsFirst／JDK 17 での実行結果

第2章

Comparator の作り方とつなぎ方

キーを取り出し、つなぎ、必要なら逆にする。

Comparator のメソッド

メソッド	種類
comparing(キー[, 比較器])	static
comparingInt・Long・Double	static
naturalOrder()・reverseOrder()	static
nullsFirst(c)・nullsLast(c)	static
thenComparing(…)	default
reversed()	default
compare(a, b)	抽象

```
Comparator<String> c = Comparator.naturalOrder();
c.reverseOrder();          // エラー
// illegal static interface method call
c.reversed();              // ○ default
Comparator.reverseOrder(); // ○ 型名で呼ぶ
```

```
list.sort(Integer::compare);   // 小さい順
list.sort((a, b) -> b - a);     // 大きい順
comparing(String::valueOf)     // "10" < "2" < "3"
```

出典：API Comparator／JLS 15.12.3／JDK 17 でのコンパイルと実行の結果

つなぐ順と reversed のかかり方

["bb", "a", "ccc", "dd"] を並べる

Comparator	結果
comparing(String::length).thenComparing(reverseOrder())	[a, dd, bb, ccc]
comparing(String::length).thenComparing(naturalOrder()).**reversed()**	[ccc, dd, bb, a]（全体が逆）

Sato 30・Ito 25・Abe 30

```
comparingInt(P::age).reversed()
  .thenComparing(P::name)
// [Abe, Sato, Ito]  名前は逆にならない
```

型が推論されない

```
comparing(s -> s.length()).reversed()
// s は Object → cannot find symbol
comparing((String s) -> s.length()).reversed() // ○
```

> reversed() は「それまでにつないだ全体」を逆にする。後ろに足した thenComparing にはかからない。

出典：API Comparator.reversed・thenComparing／JLS 15.12.2・18.5.2／JDK 17 でのコンパイルと実行の結果

ソートの道具と、安定なソート

呼び方	順
list.sort(null)	自然順序
list.sort(String.CASE_INSENSITIVE_ORDER)	大小文字を区別しない
Arrays.sort(a, 1, 3)	添字 1‑2 だけ
Arrays.sort(int[], Comparator)	コンパイルエラー
List.of(…) を sort	UnsupportedOperationException
Arrays.asList(…) を sort	できる（set だけ使う）

安定（stable）：同じ順位は元の順を保つ

```
[b, C, a, B].sort(null)
// [B, C, a, b]
.sort(CASE_INSENSITIVE_ORDER)
// [a, B, b, C]  B と b は元の順
```

TreeSet に Comparator

```
new TreeSet<>(comparing(String::length))
addAll(ab, cd, e, fgh) // [e, ab, fgh]
```

出典：API List.sort（guaranteed to be stable）・Arrays.sort・String.CASE_INSENSITIVE_ORDER・TreeSet(Comparator)／JDK 17 での実行結果

第3章

Collections と Arrays の便利なメソッド

探す・数える・並べる・作る。

探す・数える

binarySearch（昇順の List で）

1 [0]	3 [1]	5 [2]	7 [3]

binarySearch(list, 5)	2
binarySearch(list, 4)（挿入位置 2）	−2 − 1 = −3
binarySearch(list, 9)（挿入位置 4）	−4 − 1 = −5

並んでいない List では結果は保証されない。

メソッド	例
frequency(list, "a")	a の個数
disjoint(c1, c2)	共通なし → true
max(list)	自然順序の最大
min(list, reverseOrder())	自然順序の最大

[3, 1, 2] を reverse → [2, 1, 3]（逆に並べるだけ）。大きい順は sort(list, reverseOrder())。

出典：API Collections.binarySearch・frequency・disjoint・max・min・reverse／JDK 17 での実行結果

並べる・作る・包む

```
list = [a, b, c]
Collections.swap(list, 0, 2);     // [c, b, a]
Collections.addAll(list, "d", "e"); // [c, b, a, d, e]
Collections.fill(list.subList(0, 2), "x");
                       // [x, x, a, d, e]
Collections.nCopies(3, "z");      // [z, z, z]
```

メソッド	返すもの
emptyList()	空の変更できない List（add で UOE）
unmodifiableList(l)	読み取り専用の**ビュー**
synchronizedList(l)	同期化した**ビュー**

> ビューは元とつながる。syn に add した "c" は un からも見える。

出典：API Collections.swap・addAll・fill・nCopies・emptyList・unmodifiableList・synchronizedList、List.subList／JDK 17 での実行結果